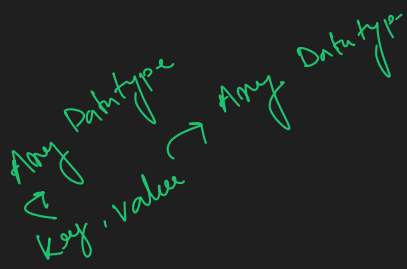


Hashing.



- insert
- remove
- search

$O(1)$

remove(key) $\rightarrow$ o/p : value.

hom. put ("Priyam", 80) ;
hom. get (" ") ;

```

hmm.size() ;
hmm.isEmpty() ;
hmm.clear() ;

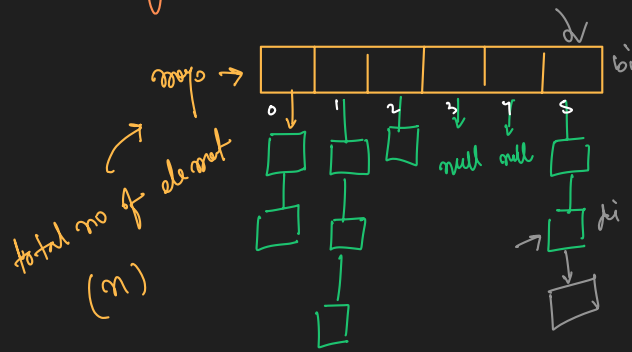
```

map, keySet () :

```
for (String key : map.keySet()) {
    sort(key);
}
```

Implementation

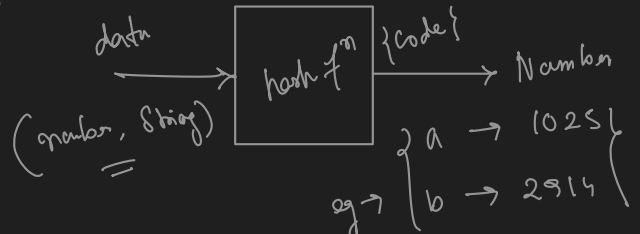
hash map $\rightarrow$ Array of Linked List



bucket size = $N \neq 6$

$\lambda = \frac{n}{N}$

hash function $\rightarrow$ hashCode()



We always have to make sure that $\lambda \leq k$ [Threshold value]
 $\{2 \text{ or } 3\}$

initially $N=4$ $\boxed{k=2} \therefore \lambda \leq 2$

$n=1, \lambda = \frac{1}{4} < 2$

$n=2, \lambda = \frac{2}{4} < 2$

$\vdots$
 $n=8, \lambda = \frac{8}{4} \leq 2$

$n=9, \lambda = \frac{9}{4} > 2$

$N=8$ $n=9, \lambda = \frac{9}{8} < 2$

key $\rightarrow$ program $\rightarrow$ 1011 $\div N \rightarrow (0, N-1)$

✓ for a particular bucket size (N) $\rightarrow$ hash function

for each key there will be a particular bucket idx. (bi)

new bucket size = $2 \times$ old bucket size
 $[N_1 = 2N]$

(key) $\rightarrow$ hash fun $\rightarrow 1011 \div N = 1011 \div 8 \rightarrow$ new bi

HashMap $\langle k, v \rangle$ map =
 $\uparrow \uparrow$
 data types

key	value
0	0


```

static class HashMap<K, V> { // Generic Class

    private class Node {

        K key;
        V value;

        public Node(K key, V value) {
            this.key = key;
            this.value = value;
        }
    }

    private int N; // Bucket Size
    private int n; // total no. of element is the map

    private LinkedList<Node> bucket[]; // N

    @SuppressWarnings("unchecked")
    public HashMap(){
        this.n = 0;
        this.N = 4;
        this.bucket = new LinkedList[N];

        for(int i=0; i<bucket.length; i++){
            bucket[i] = new LinkedList<>();
        }
    }

    public void put(K key, V value){
        int bi = hashFunction(key); // 0 to N-1
        int di = serachINLL(key, bi);

        if(di != -1){
            // update
            Node node = bucket[bi].get(di);
            node.value = value;
        }else{ // -1
            // add new node
            bucket[bi].add(new Node(key, value));
            n++;
        }

        double lambda = (double)n/N;
        if(lambda > 2.0){
            rehash();
        }
    }

    public boolean containsKey(K key){
        int bi = hashFunction(key);
        int di = serachINLL(key, bi);

        if(di != -1){
            return true;
        }else{
            return false;
        }
    }

    public V get(K key){
        int bi = hashFunction(key);
        int di = serachINLL(key, bi);

        if(di != -1){
            Node node = bucket[bi].get(di);
            return node.value;
        }else{
            return null;
        }
    }

    public V remove(K key){
        int bi = hashFunction(key);
        int di = serachINLL(key, bi);

        if(di != -1){
            Node node = bucket[bi].remove(di);
            n--;
            return node.value;
        }else{
            return null;
        }
    }
}

```

```

    private int hashFunction(K key){
        return Math.abs(key.hashCode()) % N;
    }

    private int serachINLL(K key, int bi){
        LinkedList<Node> ll = bucket[bi];

        for(int i=0; i<ll.size(); i++){
            Node node = ll.get(i);
            if(node.key == key){
                return i;
            }
        }

        return -1;
    }

    @SuppressWarnings("unchecked")
    private void rehash(){
        LinkedList<Node>[] oldBucket = bucket;

        bucket = new LinkedList[2*N]; // new bucket
        N = 2*N;

        for(int i=0; i<bucket.length; i++){
            bucket[i] = new LinkedList<>();
        }

        for(int i=0; i<oldBucket.length; i++){
            LinkedList<Node> ll = oldBucket[i];

            for(int j=0; j<ll.size(); j++){
                Node node = ll.get(j);
                put(node.key, node.value);
            }
        }
    }
}

```

```

public boolean isEmpty(){
    return n==0;
}

public List<K> keySet(){
    List<K> keys = new ArrayList<>();

    for(int i=0; i<bucket.length; i++){
        LinkedList<Node> ll = bucket[i];
        for (Node node : ll) {
            keys.add(node.key);
        }
    }

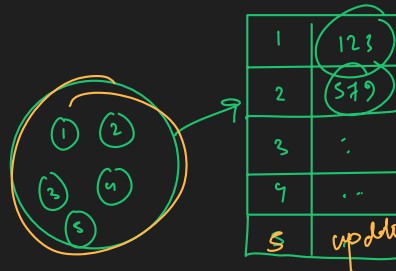
    return keys;
}

```

day-42

Hash Set → implemented by Hash Map. (key, value)
 ↓
 unique key.

→ { 1, 2, 3, 4, 5 }



HashSet<Integer> hs = new HashSet<>();

hs.add(key)

hs.contains(key) → T/F

hs.size()

hs.remove(key)

hs.isEmpty()

hs.clear()

it = it.next()

Iterator ← Iterator Interface

① Iterator<Integer> it = hs.iterator()

null 1 12 5 2 8 9 null
 ↑ ↑ ↑ ↑ ↑ ↑ ↑
 it it it it it it it

while (it.hasNext()) {
 T/F
 print(it.next()); 1, 12, 5, 2, 8, 9
}

Question 1

Majority Element

nums[] = { 1, 3, 2, 5, 1, 3, 1, 5, 1, 5, 5 }

find those element with frequency $\lceil n/3 \rceil = \lceil 11/3 \rceil = 4$

n = nums.length.
 n = 11

Ans: 1, 5

map.getOrDefault(key, default-value) → 0
 map.getOrDefault(3, 0) = 2
 map.getOrDefault(4, 0) = 0

1	1 2 3 4
3	1 2
2	1
5	1 2 3 4

hash Map (ele, freq)
 key ↑ value ↑

```

// majority Element
int nums[] = {1, 3, 2, 5, 1, 3, 1, 5, 1, 5};
HashMap<Integer, Integer> map = new HashMap<>();

// for(int num : nums){
//     if(map.containsKey(num)){
//         map.put(num, map.get(num)+1);
//     }else{
//         map.put(num, 1);
//     }
// }

for(int num : nums){
    map.put(num, map.getOrDefault(num, defaultValue: 0)+1);
}

for(Integer ele : map.keySet()){
    if(map.get(ele) >= nums.length/3){
        System.out.println(ele);
    }
}

```

Question 2

Valid Anagram

s = "race"
t = "care"

Ans = True

s = "rat"
t = "can"

Ans : False

An anagram is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all original letters exactly once.

s = ~~dog~~ ~~cat~~
t = ~~cat~~ ~~dog~~

for (int i=0 to t.length) {
 char c = t.charAt(i)
 if (c is not in hs)
 return false;
}

a	x 2/3
n	1 0
g	1 0
r	1 0
m	1 0

→ Empty()

→ True

hash map (char, freq) → hs is not Empty
↳ False

}

return hs.isEmpty();

```

public static boolean isAnagram(String s, String t){
    HashMap<Character, Integer> hs = new HashMap<>();

```

```

    for(int i=0; i<s.length(); i++){
        char c = s.charAt(i);
        hs.put(c, hs.getOrDefault(c, defaultValue: 0)+1);
    }

```

```

    for(int i=0; i<t.length(); i++){
        char c = t.charAt(i);
        if(!hs.containsKey(c)){
            return false;
        }else{
            hs.put(c, hs.get(c)-1);
            if(hs.get(c)==0){
                hs.remove(c);
            }
        }
    }

```

```

    return hs.isEmpty();
}

```

s = rat

t = can

c is not in hs

return False

r	1
a	1
t	1

H.W

Count Distinct Element

Question 3

arrays = { 4, 3, 2, 5, 6, 7, 3, 4, 2, 1 }

↓

Set = { 4, 3, 2, 5, 6, 7, 1 } → return Set.size()

Question 4

find Union and Intersection of two sets

Arr 1 = { 7, 3, 9 }

Arr 2 = { 6, 3, 9, 2, 9, 4 }

Set → A =



Set → B =



```
// Union and Intersection
int arr1[] = {7, 3, 9};
int arr2[] = {6, 3, 9, 2, 9, 4};

HashSet<Integer> A = new HashSet<>();
HashSet<Integer> B = new HashSet<>();

for(int i : arr1){
    A.add(i);
}

for(int j : arr2){
    B.add(j);
}

// System.out.println(A);
ArrayList<Integer> intersection = new ArrayList<>();
for(Integer j : B){
    if(A.contains(j)){
        intersection.add(j);
    }
}

System.out.println(intersection);

// Union
for(int j : arr2){
    A.add(j);
}
System.out.println(A);
```

Questions

Find Itinerary from tickets

Chennai → Bangalore

Mumbai → Delhi

Goa → Chennai

Delhi → Goa

Ans: Mumbai → Delhi → Goa → Chennai → Bangalore

src = ?

from to

A → B

B → C

to from

B ← A

C ← B

✓ Chennai	Bangalore
X Mumbai	Delhi
✓ Goa	Chennai
✓ Delhi	Goa

from to map

Bangalore	Chennai
✓ Delhi	Mumbai
✓ Chennai	Goa
✓ Goa	Delhi

to from map

key of from to map if not present in
key set of to from map → then that key is src

```
String tickets[][] = { // from to
    {"Chennai", "Bangalore"},
    {"Mumbai", "Delhi"},
    {"Goa", "Chennai"},
    {"Delhi", "Goa"}
};
```

```
HashMap<String, String> fromtoMap = new HashMap<>();
HashMap<String, String> tofromMap = new HashMap<>();
```

```
for(int i=0; i<tickets.length; i++){
    fromtoMap.put(tickets[i][0], tickets[i][1]);
    tofromMap.put(tickets[i][1], tickets[i][0]);
}
```

```
String src = null;
for(String stop : fromtoMap.keySet()){
    if(!tofromMap.containsKey(stop)){
        src = stop;
        break;
    }
}
```

```
System.out.print(src);
```

```
while (!fromtoMap.isEmpty()) {
    String dest = fromtoMap.remove(src);
    System.out.print("-->" + dest);
    src = dest;
}
```

```
System.out.println();
```